

Cómo ejecutar y probar Delegaciones Municipales

Para ejecutar los 15 casos del informe con HTTPS y cierre por inactividad, usar [Pruebas del informe y capturas](pruebas-del-informe.md). Esa guía incluye instalación del certificado público local. Los comandos HTTP siguientes se conservan como opción de desarrollo. La sesión ahora expira tras 30 minutos de inactividad y el navegador redirige automáticamente al acceso.

Esta guía permite probar el prototipo en el computador del usuario. No requiere que el repositorio de GitHub ya contenga estos cambios: se utiliza el ZIP entregado con el código actual. La dirección local del computador no permite acceder a una aplicación que corre en otra máquina cloud.

1. Preparar el computador

En Windows o macOS, instalar y abrir Docker Desktop, con contenedores Linux. En Linux se puede usar Docker Engine con el complemento Docker Compose. Se necesita conexión a Internet la primera vez para descargar las imágenes de Python y PostgreSQL.

Se recomienda disponer de aproximadamente 2 GB libres y 2 GB de RAM para esta demostración. La imagen se verificó en Linux x86_64. El ZIP incluye paquetes Python para Linux x86_64 y ARM64; la ejecución en Windows, macOS o ARM64 no fue comprobada físicamente en este entorno.

Descargar `Delegaciones-Municipales-demo.zip`, extraerlo y abrir una terminal dentro de la carpeta `Delegaciones-Municipales`. En Windows se puede abrir PowerShell desde esa carpeta. Verificar:

```
```sh
docker --version
docker compose version
```
```

Si Docker no responde, abrir Docker Desktop y esperar hasta que indique que el motor está funcionando. Usar Compose v2; el comando es `docker compose`, con espacio.

2. Iniciar el sistema

Desde la carpeta que contiene `compose.yaml`, ejecutar:

```
```sh
docker compose up --build -d
docker compose ps
```
```

La primera construcción puede tardar varios minutos. Los servicios `web` y `db` deben aparecer como `healthy`. El servicio `init` finaliza con código 0 tras preparar la configuración; no debe mantenerse ejecutándose.

Docker prepara la base de datos, aplica migraciones y crea dos delegaciones, cuatro cuentas y un vecino ficticio. Las contraseñas se generan al azar y los datos se guardan en volúmenes persistentes. El servicio web es de desarrollo, apropiado para esta demostración, no para publicar con datos personales reales.

3. Elegir las contraseñas de prueba

Para acceder sin consultar archivos privados, asignar una contraseña propia a cada cuenta mediante estos comandos, uno a la vez:

```
```sh
docker compose exec web python manage.py changepassword admin.demo
docker compose exec web python manage.py changepassword delegado.norte
docker compose exec web python manage.py changepassword funcionario.norte
docker compose exec web python manage.py changepassword funcionario.sur
```
```

Cada comando solicita la contraseña dos veces. **La terminal no muestra caracteres mientras se escribe; es normal.** Usar al menos 12 caracteres y evitar contraseñas comunes. Si la validación la rechaza, elegir otra y no omitir los validadores. No pegar contraseñas en el chat ni incluirlas en el informe. Estos comandos sirven para preparar

las cuentas locales; los cambios de contraseña realizados desde la administración de la aplicación quedan registrados en su auditoría.

Se pueden utilizar contraseñas distintas por cuenta. Luego abrir en el navegador del mismo computador:

```
```text
http://127.0.0.1:8000
```
```

Esto es una dirección de la instalación local del usuario, no un enlace de preview del entorno cloud. No es necesario instalar Python, PostgreSQL o `uv` en el computador cuando se utiliza Docker.

4. Recorrido de prueba manual

Registro e ingreso de solicitud

1. Iniciar sesión como `funcionario.norte`.
2. Abrir **Vecinos → Registrar vecino**. Usar nombre ficticio, RUT de prueba `11.111.111-1`, correo `prueba@example.invalid` y dirección ficticia. Si ese RUT ya existe, utilizar el registro existente.
3. Intentar registrar el RUT `11.111.111-2`: debe rechazarse por dígito verificador incorrecto.
4. Ingresar una solicitud de tipo **Reclamo**, por ejemplo «Revisión ficticia de luminaria».
5. Anotar el número de solicitud. Debe quedar **Ingresada**. El funcionario no puede asignarla ni cerrarla.

Asignación, atención y cierre

1. Salir e ingresar como `delegado.norte`.
2. Abrir la solicitud y asignarla a `funcionario.norte`, indicando una observación. Debe quedar **Asignada**.
3. Volver a ingresar como `funcionario.norte` y avanzar a **En atención** con una observación.
4. Documentar una solución ficticia y avanzar a **Resuelta**.
5. Ingresar como `delegado.norte`, revisar la solución y autorizar el cierre. Debe quedar **Cerrada**.
6. Revisar los cuatro cambios del historial, con autores, fechas y motivos. La solicitud cerrada no debe aceptar nuevas observaciones ni volver a otro estado.

Para alternar cuentas también se pueden usar perfiles o ventanas privadas independientes. No mezclar dos roles dentro de una misma sesión del navegador.

Agenda

1. Como `funcionario.norte`, abrir **Agenda → Reservar atención**.
2. Elegir el vecino, `funcionario.norte`, una fecha futura y una duración de 30 minutos. La solicitud relacionada es opcional; una cerrada no estará disponible.
3. Crear otra reserva para el mismo funcionario en el mismo horario: debe rechazarse y conservar los datos del formulario.
4. Probar un horario inmediatamente posterior al fin de la primera cita: debe admitirse.
5. Cancelar la primera cita, indicando motivo. Debe aparecer en el historial y liberar el horario para una reserva nueva.
6. Intentar marcar una cita futura como atendida: debe rechazarse. Para comprobar la atención realizada, reservar pocos minutos en el futuro y esperar hasta su hora de inicio. Entonces el funcionario asignado o el delegado podrán marcarla **Atendida**, con un resultado.

Todas las fechas se muestran en horario de Chile. El prototipo no impone un horario de oficina. Las reservas del mismo funcionario no pueden superponerse; un vecino puede tener citas con funcionarios diferentes en el mismo horario.

Reportes y permisos

1. Como `delegado.norte`, abrir **Reportes**. Filtrar por estado **Cerrada** y por el período que incluya la fecha de ingreso de la solicitud.
2. Confirmar que aparece la solicitud cerrada y que los totales por estado/delegación coinciden con el detalle.
3. Probar un período con fecha final anterior a la inicial: debe mostrarse un error y no

ampliarse el reporte.

4. Como `funcionario.norte`, no debe aparecer el menú Reportes. Una petición directa a `/reportes/` debe mostrar acceso denegado.

5. Como `funcionario.sur`, abrir el detalle de la solicitud anotada (`/solicitudes/NUMERO/`): debe indicar registro no disponible. No debe ver vecinos o citas de Norte.

6. Como `admin.demo`, consultar ambas delegaciones y abrir **Auditoría**. Deben existir los eventos de ingreso, cambios de estado, reserva y cancelación.

Administración y contraseñas

Con `admin.demo`, abrir **Administración**. Se pueden gestionar usuarios y delegaciones. Para un cambio de contraseña, abrir un usuario y utilizar la opción de contraseña; los valores débiles se rechazan y los cambios exitosos dejan un evento de auditoría. La contraseña se guarda como hash y no se muestra en la auditoría.

La aplicación no permite eliminar usuarios/delegaciones desde esa interfaz, ni editar o eliminar eventos de auditoría. No permite cambiar de rol/delegación o desactivar a un funcionario con citas programadas: primero se deben finalizar o cancelar.

Accesibilidad

Probar las pantallas con teclado, usando Tab y Shift+Tab. Desde el campo de usuario del login, Shift+Tab alcanza **Saltar al contenido**. Comprobar foco visible, etiquetas y mensajes. Ajustar la ventana a aproximadamente 768 píxeles de ancho; las tablas pueden desplazarse dentro de su contenedor sin desbordar toda la página.

5. Ejecutar las pruebas automatizadas

Con `web` y `db` iniciados:

```
```sh
docker compose exec web python manage.py test apps
```
```

El resultado verificado tras los ajustes al informe y preparación EC2 es **115 pruebas y 0 OK**, sin pruebas omitidas. Se crea una base temporal de PostgreSQL y se elimina al terminar; no se borra la base utilizada por la demostración.

Los recorridos automatizados de navegador y la revisión de accesibilidad son herramientas opcionales de desarrollo y tienen requisitos adicionales descritos en README. No son necesarios para probar manualmente la aplicación con Docker Desktop.

6. Detener, reiniciar y solucionar problemas

Para detener sin borrar los datos:

```
```sh
docker compose down
```
```

Para volver a iniciar:

```
```sh
docker compose up -d
```
```

Las contraseñas y datos se conservan en volúmenes. No agregar `-v` al comando de detención si se desea conservarlos. Para incorporar cambios posteriores del código, volver a ejecutar `docker compose up --build -d`.

Si no aparece la pantalla, comprobar que `web` esté saludable y revisar:

```
```sh
docker compose ps
docker compose logs --tail=50 web
docker compose logs --tail=50 init
```
```

Si el puerto 8000 está ocupado, en PowerShell ejecutar:

```
```powershell
$env:APP_PORT="8010"
docker compose up -d
```
```

En Linux o macOS:

```
```sh
APP_PORT=8010 docker compose up -d
```
```

Después abrir `http://127.0.0.1:8010` en el navegador local. Mantener ese valor al utilizar los comandos posteriores. Si se bloquea el login tras cinco fallos, esperar una hora o, desde la terminal autorizada de esta instalación de prueba, ejecutar `docker compose exec web python manage.py axes\_reset`. La prueba de bloqueo se recomienda al final para no interrumpir el recorrido principal.

Si falla la descarga de imágenes, comprobar la conexión y la configuración de proxy de Docker Desktop. No desactivar TLS ni modificar los hashes. El ZIP contiene las dependencias Python verificadas, pero no contiene las imágenes Docker.

7. Registrar la aceptación

Guardar observaciones sobre cada recorrido: usuario/rol, acción, resultado esperado, resultado observado y captura con datos ficticios. La aceptación humana se registra solo después de que el usuario realiza estas comprobaciones. Las pruebas automáticas aprobadas no equivalen a certificación ISO, auditoría integral de producción ni cumplimiento legal total.